

HTTP 4장 커넥션 관리

이재훈

목차

- 4.1 TCP 커넥션
- 4.2 TCP의 성능에 대한 고려
- 4.3 HTTP 커넥션 관리
- 4.4 병렬 커넥션
- 4.5 지속 커넥션
- 4.6 파이프라인 커넥션
- 4.7 커넥션 끊기에 대한 미스터리

커넥션?

-연결

-http에서 클라이언트(브라우저)와 서버가 통신 → 커넥션

클라이언트-서버간 메시지 손실, 손상, 순서 변경 X, 안전하게 전달

왜 커넥션을 알아야할까?

- http 바탕에서 개발
- 복잡하면서도 빠른 애플리케이션 개발
- 최적화 요소, 더 좋은 성능의 http 애플리케이션

4.1 TCP 커넥션

인터넷 프로토콜 스택의 4계층

애플리케이션 계층 - HTTP, FTP

전송 계층 - TCP, UDP

인터넷 계층 - IP

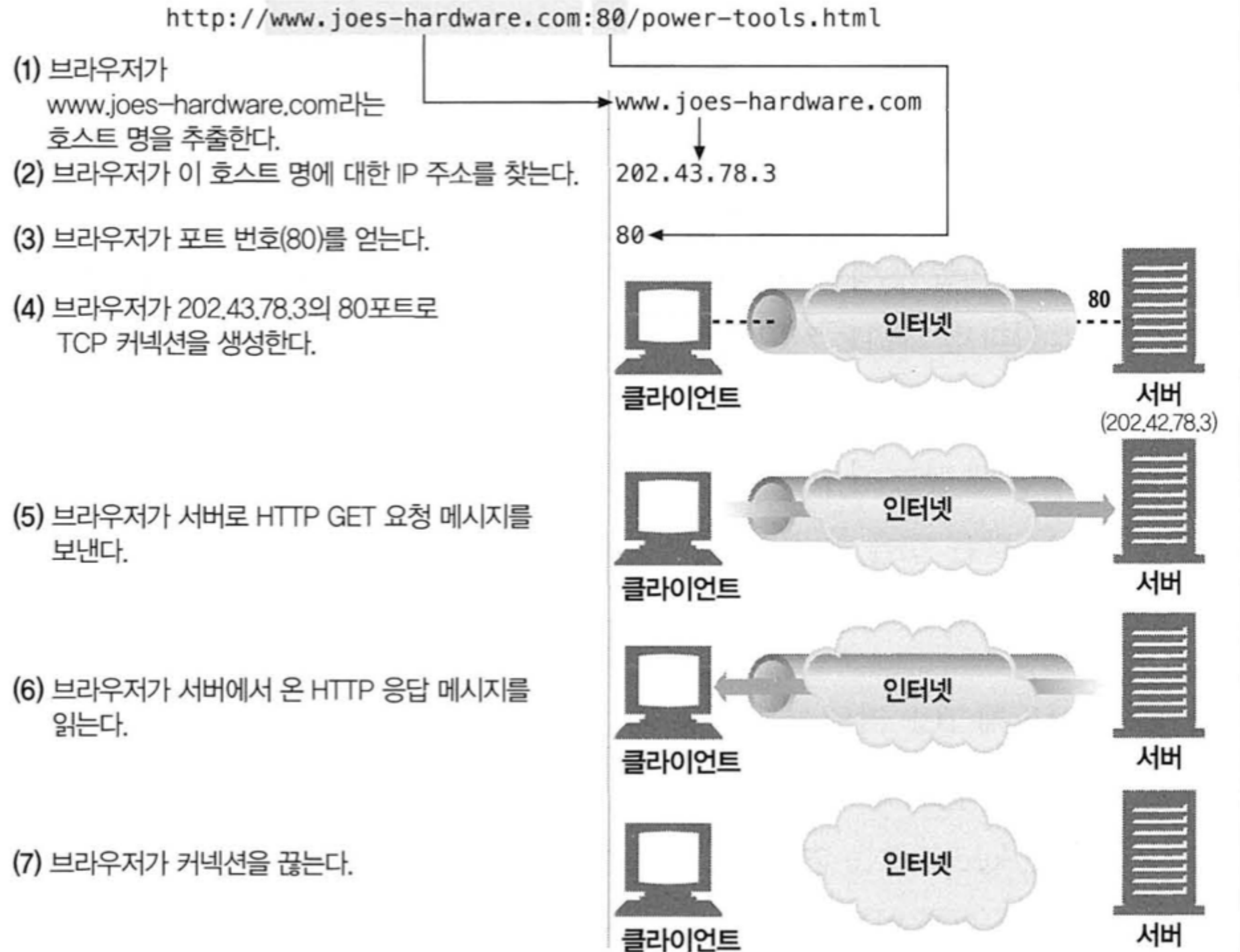
네트워크 인터페이스 계층

HTTP 통신은 계층화된 집합인 TCP/IP를 통해 이뤄진다.

HTTP 커넥션 = TCP 커넥션 + a

HTTP 커넥션 ÷ TCP 커넥션

4.1 TCP 커넥션



4.1 TCP 커넥션

TCP의 통신 방식

TCP는 IP 패킷 이용하여 데이터 전송

데이터 를 세그먼트로 잘게 나누고 IP 패킷이라는 봉투에 담아 데이터 전달.

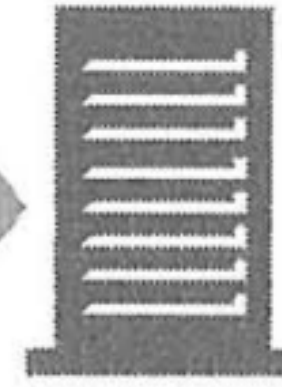


클라이언트

TCP
세그먼트
#1

TCP
세그먼트
#2

TCP
세그먼트
#3

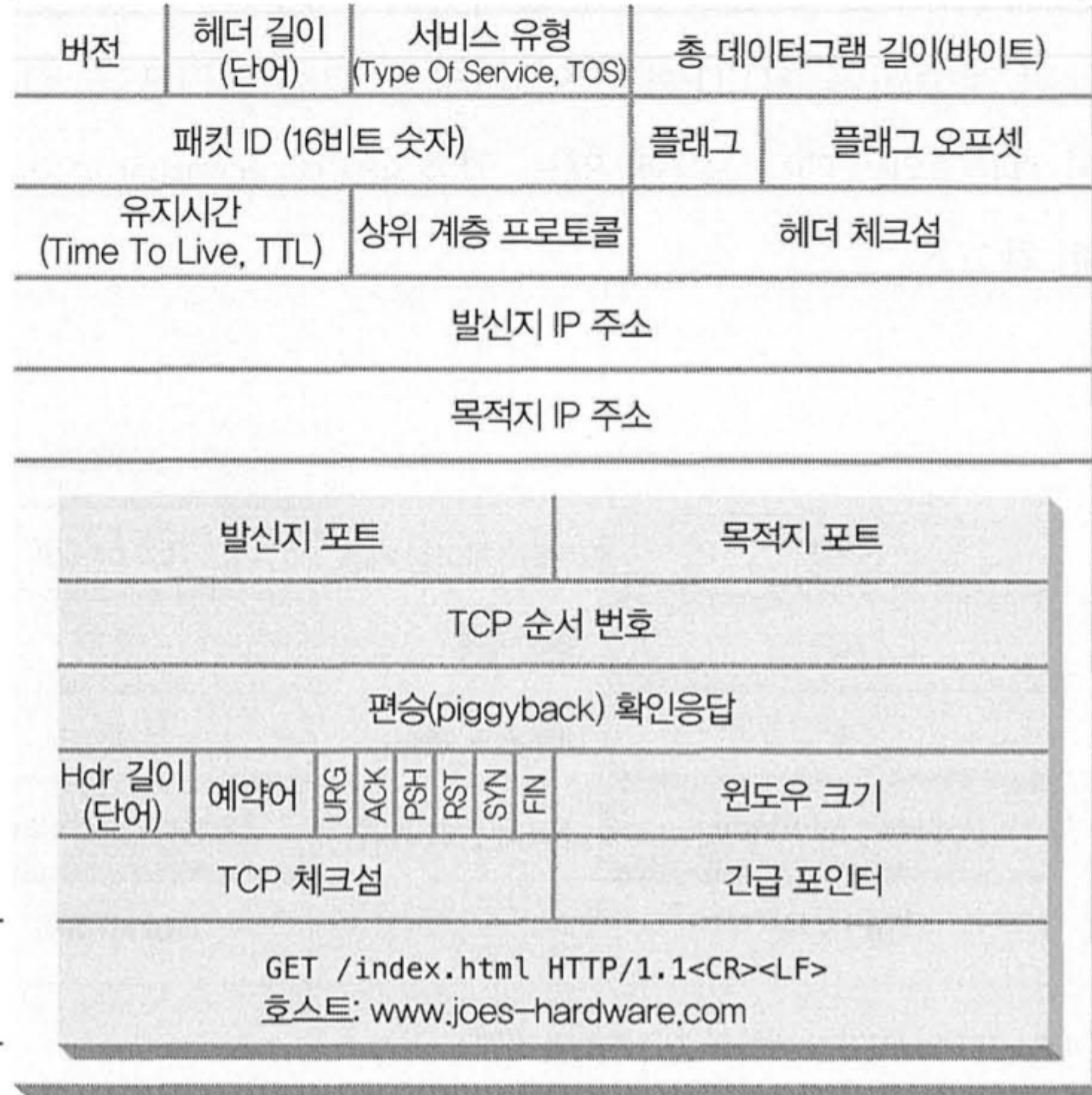


서버

IP 패킷

TCP 세그먼트

TCP 데이터 스트림 덩어리



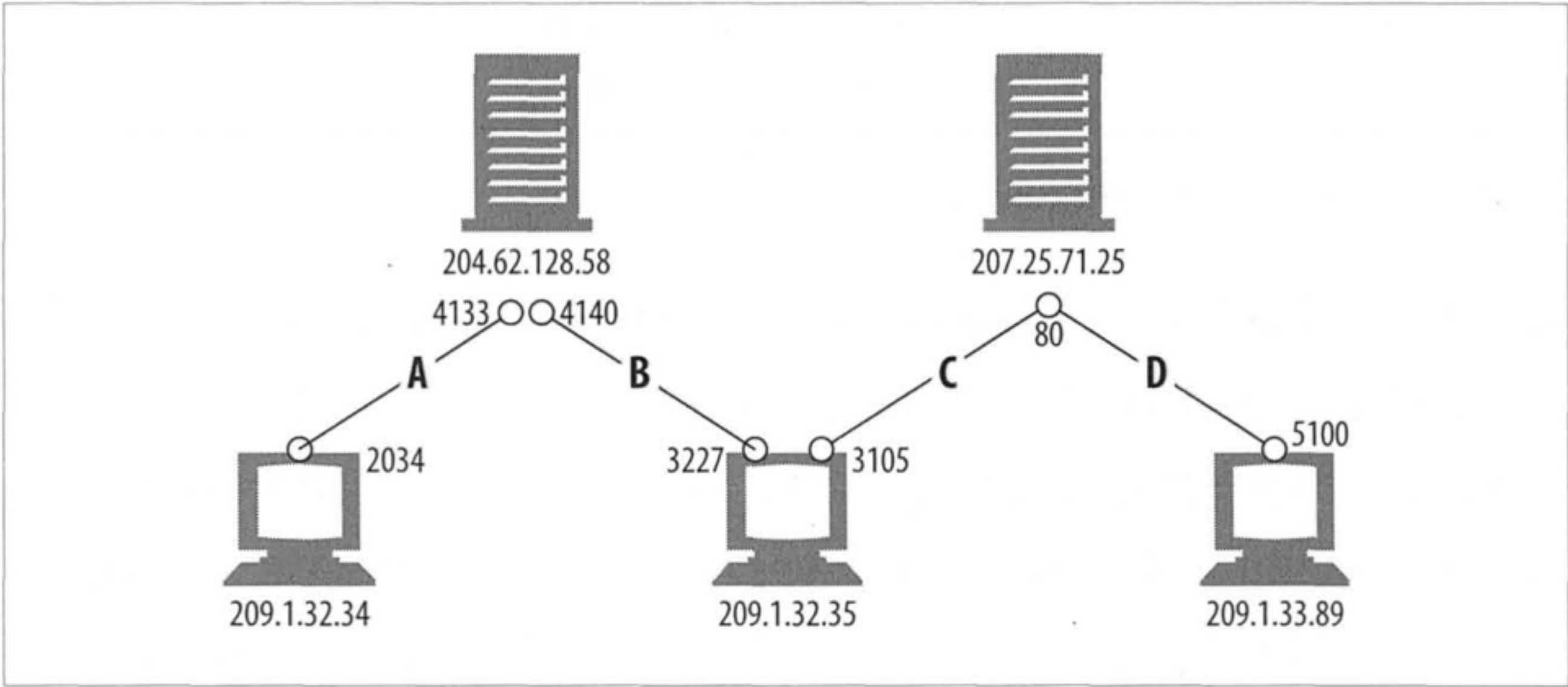
4.1 TCP 커넥션

커넥션 유지하기

- 발신지 IP주소
- 발신지 포트
- 수신지 IP주소
- 수신지 포트

커넥션	발신지 IP 주소	발신지 포트	목적지 IP 주소	목적지 포트
A	209.1.32.34	2034	204.62.128.58	4133
B	209.1.32.35	3227	204.62.128.58	4140
C	209.1.32.35	3105	207.25.71.25	80
D	209.1.33.89	5100	207.25.71.25	80

표 4-1 TCP 커넥션 값



네가지 주소 구성요소의 값이 모두 같은 순 없다.

4.1 TCP 커넥션

TCP 소켓 프로그래밍

-IP 주소와 포트만 있으면 통신 가능?

✗ 주소만으론 안 되고, OS에게 “통신하겠다”는 소켓 설정이 필요

4.1 TCP 커넥션

TCP 소켓 프로그래밍

소켓 API 호출	설명
<code>s = socket(<parameters>)</code>	연결이 되지 않은 익명의 새로운 소켓 생성
<code>bind(s, <local IP:port>)</code>	소켓에 로컬 포트 번호와 인터페이스 할당
<code>connect(s, <remote IP:port>)</code>	로컬의 소켓과 원격의 호스트 및 포트 사이에 TCP 커넥션 생성
<code>listen(s,...)</code>	커넥션을 받아들이기 위해 로컬 소켓에 허용함을 표시
<code>s2 = accept(s)</code>	누군가 로컬 포트에 커넥션을 맺기를 기다림
<code>n = read(s,buffer,n)</code>	소켓으로부터 버퍼에 n바이트 읽기 시도
<code>n = write(s,buffer,n)</code>	소켓으로부터 버퍼에 n바이트 쓰기 시도
<code>close(s)</code>	TCP 커넥션을 완전히 끊음
<code>shutdown(s, <side>)</code>	TCP 커넥션의 입출력만 닫음
<code>getsockopt(s, ...)</code>	내부 소켓 설정 옵션값을 읽음
<code>setsockopt(s, ...)</code>	내부 소켓 설정 옵션값을 변경

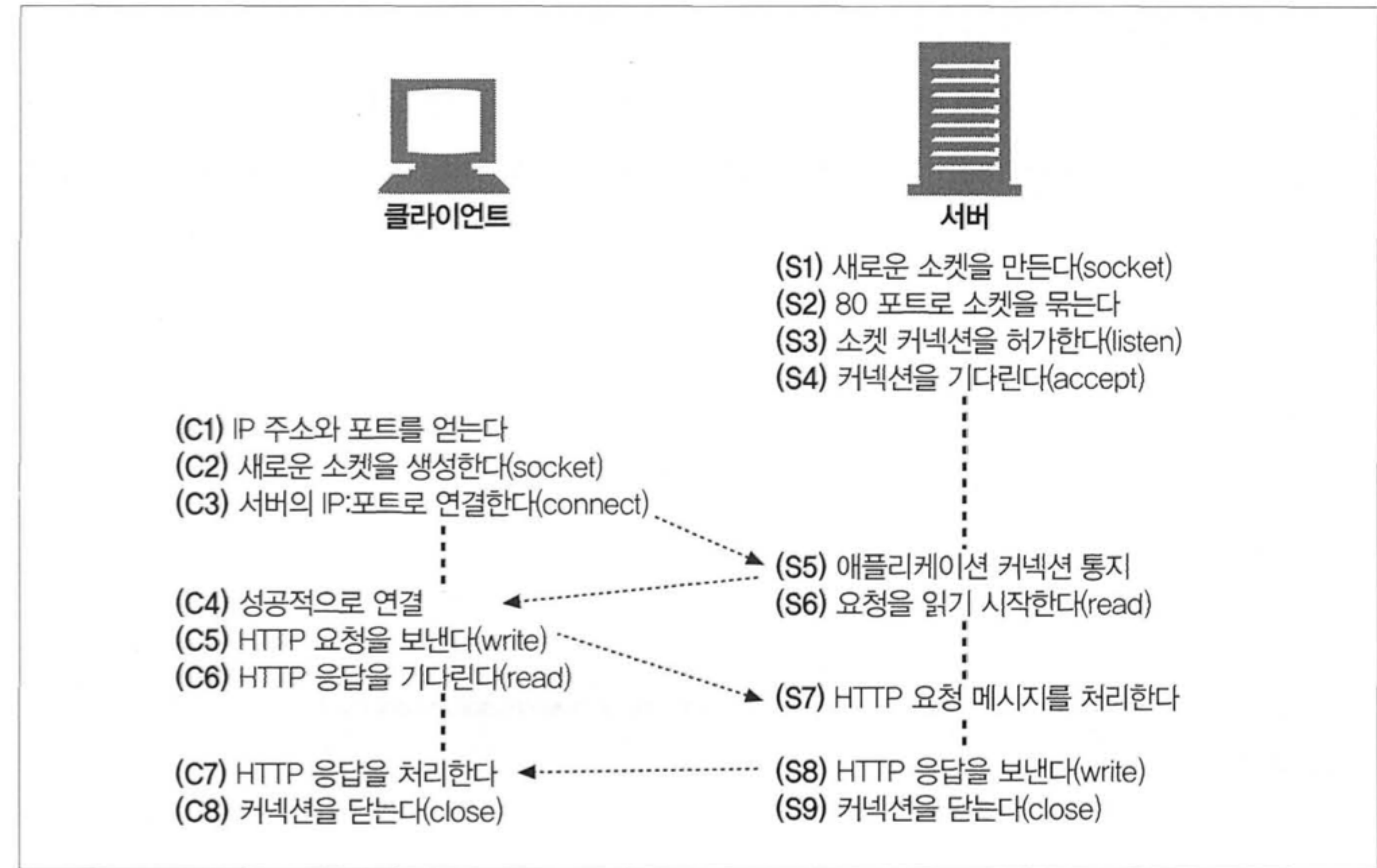


그림 4-6 클라이언트와 서버가 TCP 소켓 인터페이스를 사용하여 상호작용하는 방법

4.1 TCP 커넥션

4.2 TCP의 성능에 대한 고려

HTTP는 TCP 바로 위에 있는 계층

HTTP 트랜잭션 성능 <- TCP 성능

TCP 이해 -> 커넥션 최적화 -> 더 좋은 성능으로 개발

4.2 TCP의 성능에 대한 고려

트랜잭션 지연

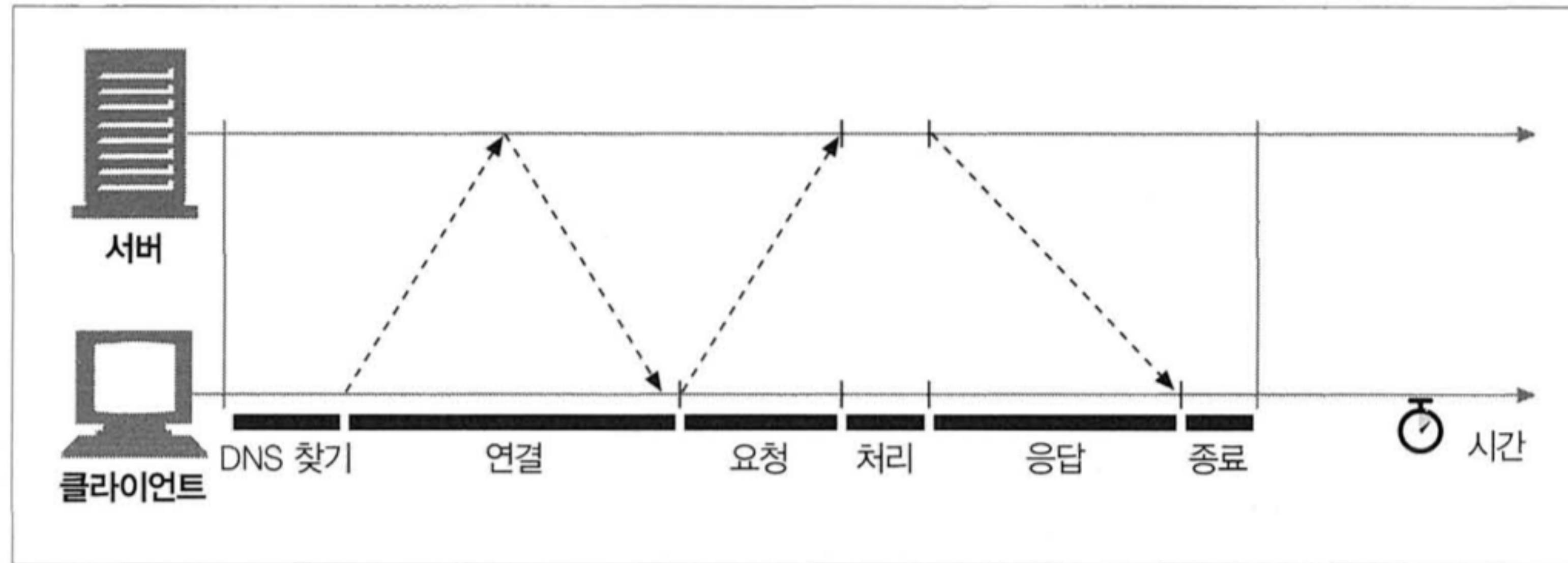


그림 4-7 HTTP 트랜잭션이 처리되는 과정

4.2 TCP의 성능에 대한 고려

- TCP 커넥션의 핸드셰이크 설정
- 인터넷의 혼잡을 제어하기 위한 TCP의 느린 시작(slow-start)
- 데이터를 한데 모아 한 번에 전송하기 위한 네이글(nagle) 알고리즘
- TCP의 편승(piggyback) 확인응답(acknowledgment)을 위한 확인응답 지연 알고리즘
- TIME_WAIT 지연과 포트 고갈

4.3 HTTP 커넥션 관리

Connection 헤더

HTTP 통신에서 “이 연결(커넥션)에만 적용되는 옵션이나 정보”를 지정하는 데 사용되는 헤더 왜 필요할까?

클라이언트와 서버 사이 여러 프락시 - 캐시 서버 등 중간 서버 존재

Ex) `Connection: close`는 “이 연결이 끝나면 닫아라”는 의미인데, 중간 서버가 이것을 다음 서버로 전달 ❌

Connection 헤더에 명시된 헤더들 “hop-by-hop(홉별)” 헤더

4.3 HTTP 커넥션 관리

Connection 헤더

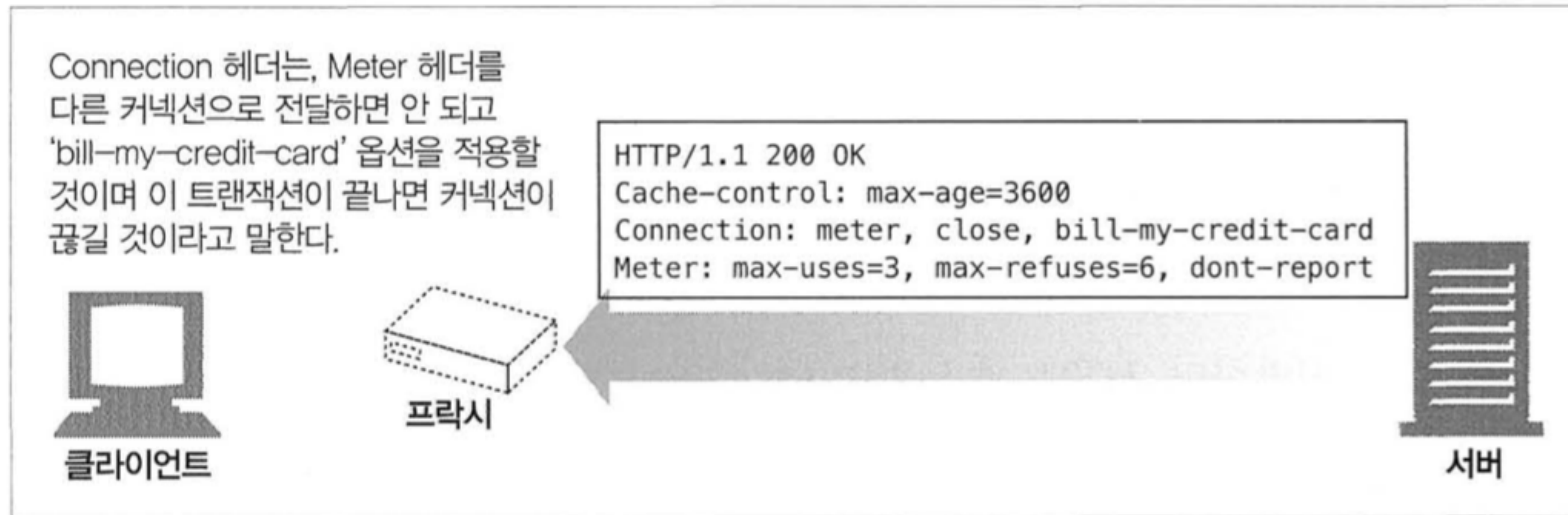
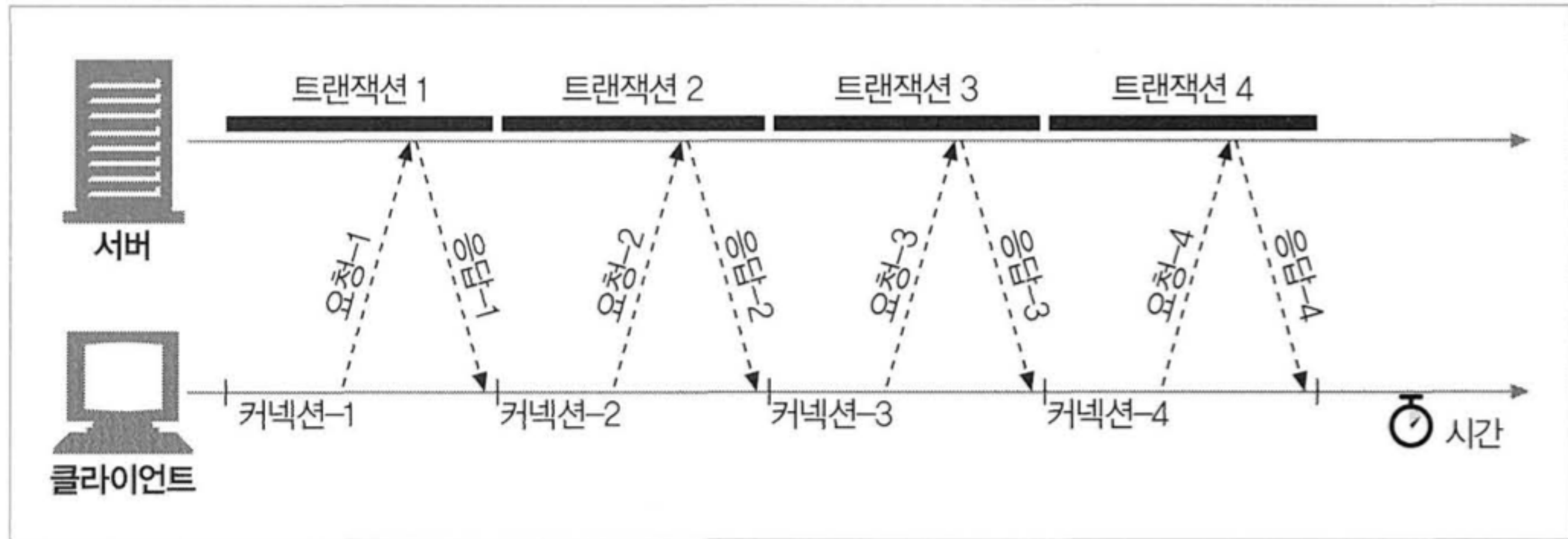


그림 4-9 Connection 헤더는 전송자가 특정 커넥션에만 해당되는 옵션을 지정하게 해준다.

4.3 HTTP 커넥션 관리

순차적인 트랜잭션 처리 지연



4.3 HTTP 커넥션 관리

순차적인 트랜잭션 처리 지연

-병렬 커넥션

-지속 커넥션

-파이프라인 커넥션

-다중 커넥션

4.4 병렬 커넥션

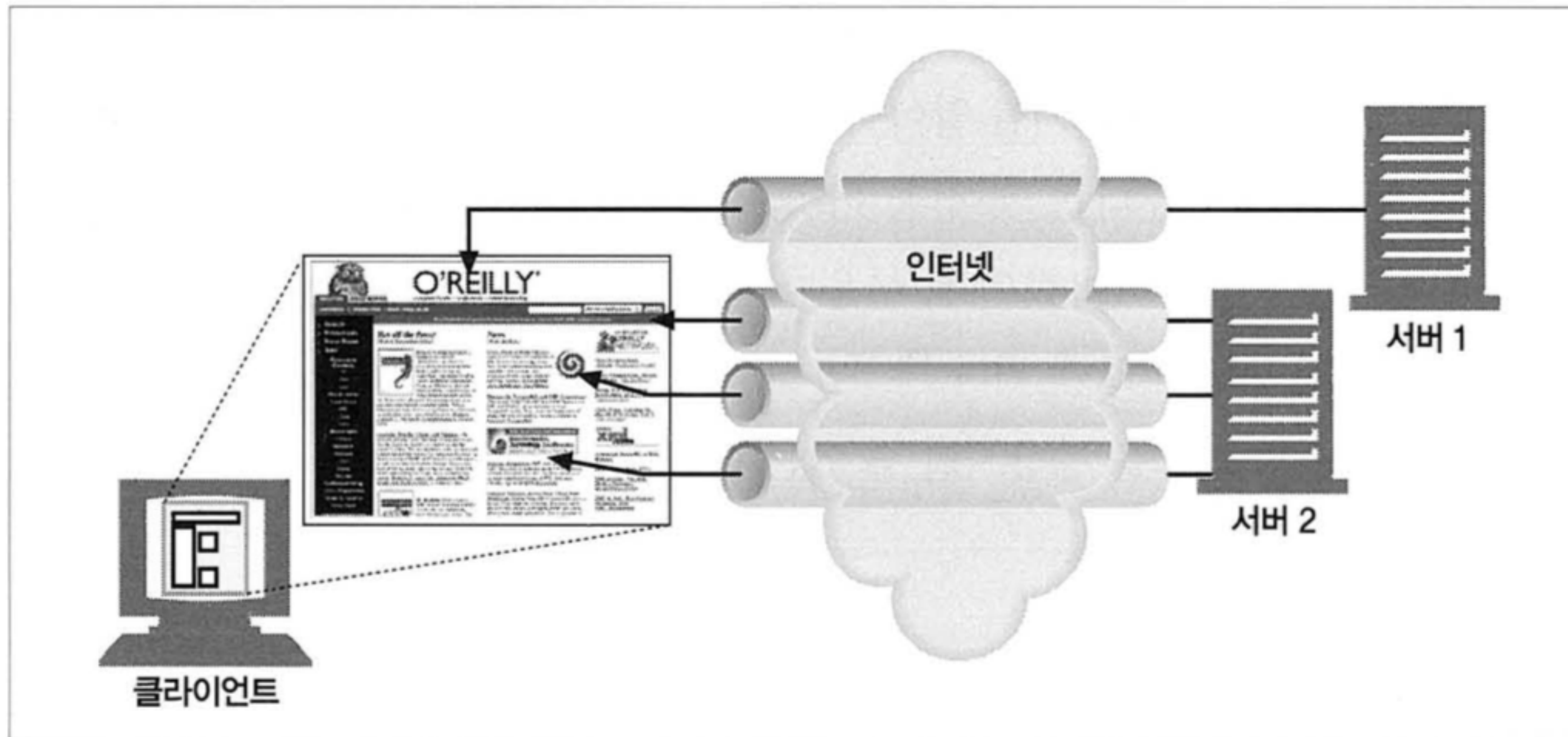


그림 4-11 웹페이지의 컴포넌트들은 각각의 HTTP 커넥션에서 처리된다.

빠르지만은 항상 빠른 건 ✖

-대역폭이 좁을 때

성능상 장점 ✖

한 서버에 다수의 사용자가

병렬 커넥션 사용한다면?

메모리, 성능문제

4.5 지속 커넥션

HTTP/1.1 TCP 커넥션 유지 재사용

처리 완료 후에도 'close' 하지 않고 유지

장점은?

-커넥션 사전 작업과 지연

단점은?

-계속 연결된 상태로 쌓임 주의

4.5 지속 커넥션

지속 커넥션 vs 병렬 커넥션

두 가지 함께 사용할 때가 가장 효과적

4.5 지속 커넥션

지속 커넥션 vs 병렬 커넥션

두 가지 함께 사용할 때가 가장 효과적

4.5 지속 커넥션

HTTP/1.0+ keep alive 커넥션, HTTP/1.1 지속 커넥션

아직 많은 클라이언트, 서버는 초기 keep-alive 커넥션 사용

클라이언트가 Connection: keep-alive 헤더를 추가



Connection: Keep-Alive
Keep-Alive: max=5, timeout=120

그림 4-14 HTTP/1.0 keep-alive 트랜잭션 헤더 핸드셰이크

4.5 지속 커넥션

HTTP/1.0+ keep alive 커넥션, HTTP/1.1 지속 커넥션

HTTP/1.0+에서 keep alive는 기본적으로는 사용❌

—>매번 Connection: keep-alive 헤더를 추가.

4.5 지속 커넥션

Keep-Alive와 멍청한 프락시

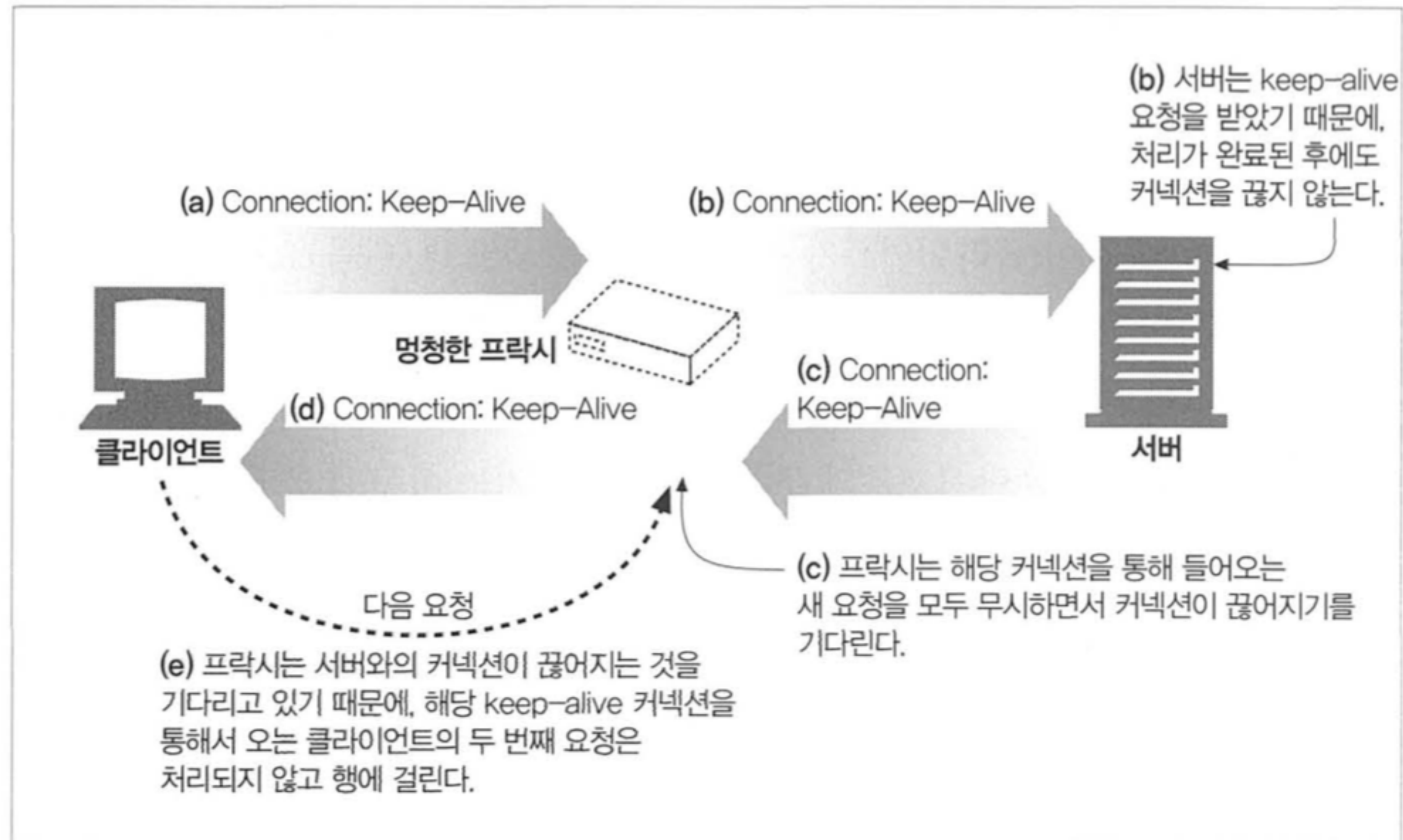


그림 4-15 keep-alive는 Connection 헤더를 지원하지 않은 프락시와는 상호작용하지 않는다.

4.5 지속 커넥션

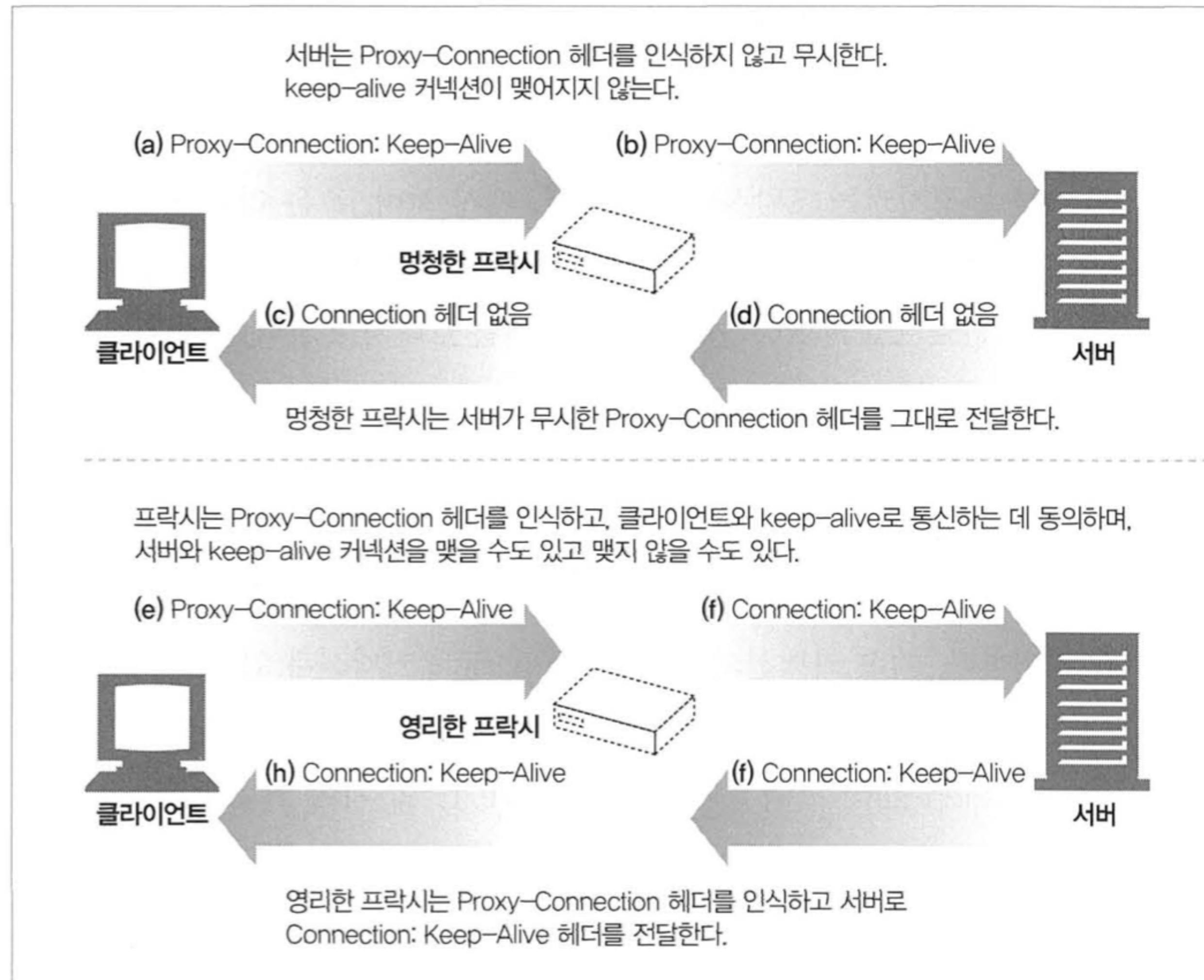


그림 4-16 Proxy-Connection 헤더는 단일 무조건 전달 문제를 해결해준다.

4.5 지속 커넥션

HTTP/1.0+ keep alive 커넥션, HTTP/1.1 지속 커넥션

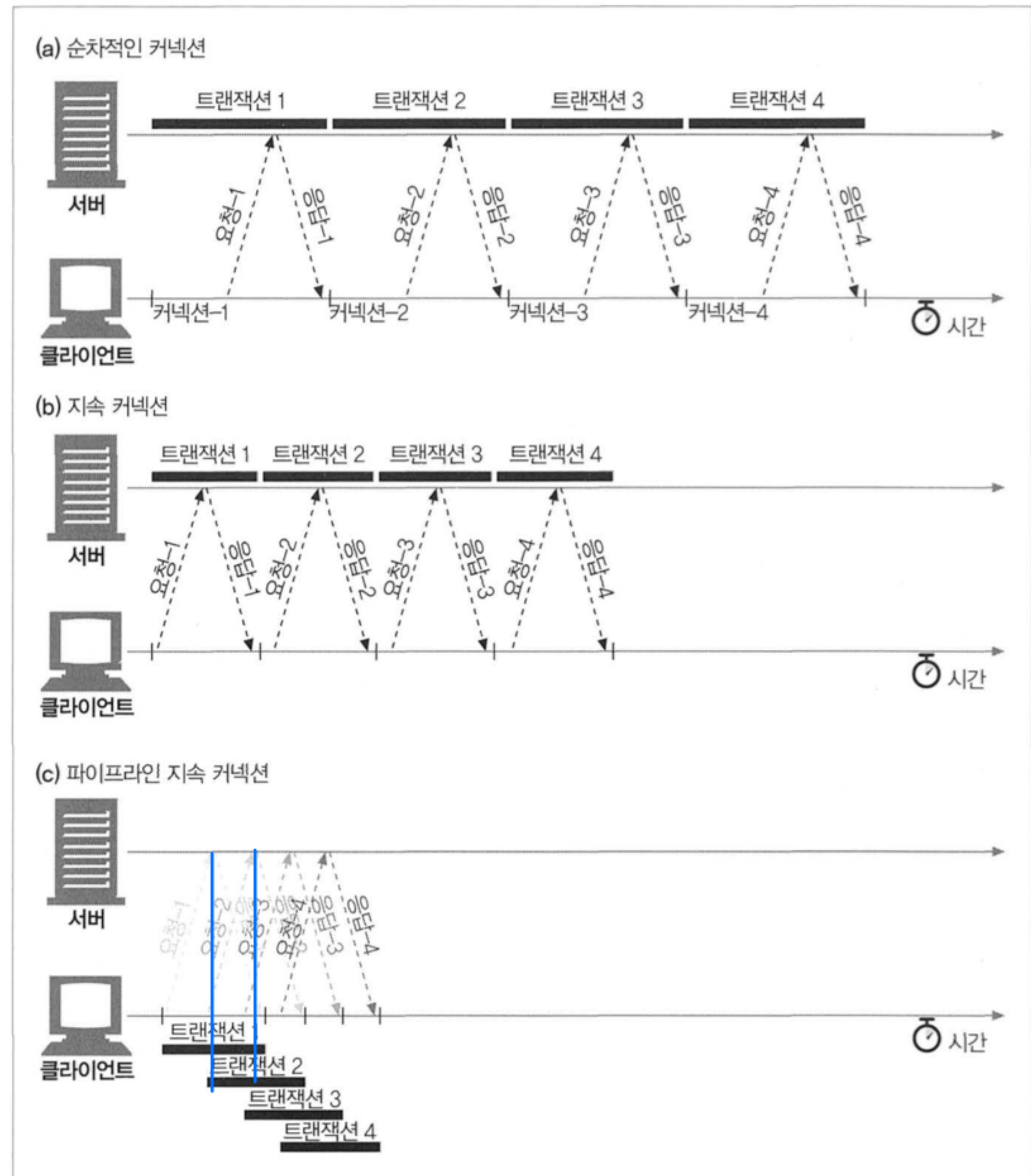
HTTP/1.1의 지속 커넥션은 기본으로 활성화 <-> HTTP/1.0의 keep-alive

Connection: close 헤더가 없으면 계속 유지. 하지만 언제든지 끊을 순 있음.
->영원 ❌

4.6 파이프라인 커넥션

지속커넥션을 통해...

- 지속커넥션인지 확인전에는 파이프라인❌
- HTTP 응답은 요청 순서와 같게
- 언제든 다시 요청을 보낼 준비
- Post와 같은 비먹등 메서드 사용 주의



4.7 커넥션 끊기에 대한 미스터리

언제 어떻게 끊는가?...

명확한 기준이 없음.